That last line shows that the empty hash is treated differently—an empty hash only matches another empty hash.

You can use * at the end of an array pattern to indicate rest, and you can use one at the front to indicate a "find pattern", where you are looking for an element in the middle.

```
[1, "potato", 2, "potato"] in [Integer, "potato", Integer, "potato"] # => true
[1, "potato", 2, "potato"] in [Integer, "potato", *]                 # => true
[1, "potato", 2, "potato"] in [*, "potato", 2, *]                    # => true
```

You can nest the data:

```
{likes: [3, 5], dislikes: [2, 4]} in {likes: [3, *], dislikes: [2, *]} # => true
```

And you can provide multiple patterns using "or" logic:

```
[1, 2, 3] in [Integer, Integer, Integer] | [String, String, String]       # => tr
                                                                          # .. ue
["a", "b", "c"] in [Integer, Integer, Integer] | [String, String, String] # => tr
                                                                          # .. ue
["a", "b", 3] in [Integer, Integer, Integer] | [String, String, String]   # => fa
                                                                          # .. ls
                                                                          # .. e
```

The last example is false because the left side of the expression does not match either of the patterns completely.

Variable Binding

Where this starts to get really powerful is that you can also assign values in the target to variables in the pattern, and then use those values. You can include a bare variable in the pattern by adding a hashrocket => and a local variable name to any part of a pattern:

```
"value" in String => a
puts a
```

produces:

```
value
```

Note that the in expression still returns true, the variable assignment is a side effect.

If you just want the variable assignment and don't care about the truth value, you can replace the in operator with =>, which we have already seen can be used for rightward assignment:

```
"value" => String => a
puts a
```

produces:

```
value
```

There's a shortcut if the part of the pattern being assigned doesn't have any other pattern matching syntax, you can leave off the hashrocket and just put in the local name:

```
"value" in a
puts a

"Another value" => b
puts b
```